ENTERPRISE SECURITY ASSESSMENT LAB

API Security Evidence Documentation Report

JWT Decoding and Token Structure Analysis Evidence

Professional Portfolio Edition - Authorized API Security Lab Evidence

Field	Details
Test Area	JWT decoded in jwt.io
Result Type	Token analysis
Evidence Report File	03-jwtio-decoded-token-evidence-report.pdf
Testing Phase	JWT Structure Review
Tool / Component	jwt.io / Local JWT Decode Review
OWASP / Security Mapping	API2:2023 Broken Authentication
Repository Path	02-api-security/reports/evidence/
Prepared For	GitHub recruiter portfolio documentation

This report documents one API Security evidence row as a standalone professional artifact. It includes the embedded screenshot, what it proves, the methodology context, command/workflow, sanitized output style, risk interpretation, remediation guidance, and publication redaction requirements.

1. Embedded Evidence Screenshot

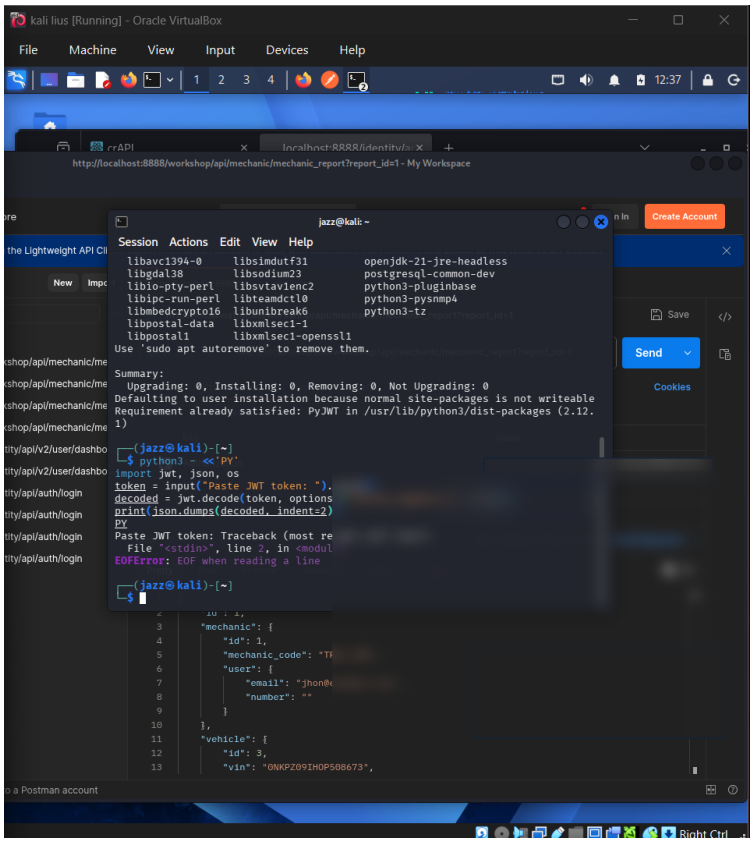

Figure 1 - Embedded screenshot evidence for: JWT decoded in jwt.io

What this evidence proves

The evidence shows token review activity. Sensitive token material and selected values are blurred, which is correct because decoded tokens may expose usernames, IDs, roles, and session metadata.

Evidence handling note

The screenshot has been prepared for GitHub publication. Sensitive fields such as credentials, tokens, JWTs, cookies, personal data, host values, or other secrets are blurred or represented as redacted values where needed.

2. Assessment Context and Methodology Placement

Area	Value
Testing Phase	JWT Structure Review
Tool / Component	jwt.io / Local JWT Decode Review
Assessment Objective	Decode the JWT header and payload to understand token structure, algorithm selection, claims, roles, identity values, and expiry behaviour without modifying the token.
Result Type	Token analysis
OWASP / Security Mapping	API2:2023 Broken Authentication

Why this evidence matters

Decoding a JWT is analysis, not exploitation. The goal is to understand whether the token contains risky claims, weak algorithms, excessive identity data, or missing expiry controls.

Command / workflow used

```
Paste token into jwt.io or decode locally
Review header: alg, typ
Review payload: sub, role, iat, exp, user identifiers
Do not publish complete token values
```

Sanitized output style

```
JWT header: { "alg": "<REVIEWED>", "typ": "JWT" }
JWT payload: { "sub": "<REDACTED>", "role": "<REVIEWED>", "exp": "<REVIEWED>" }
```

3. Professional Interpretation

This evidence row should be interpreted as: **Token analysis**.

If JWTs contain excessive sensitive data, lack expiry, use weak signing, or are accepted after tampering, attackers may abuse them to impersonate users or escalate privileges.

Assessor notes

Professional reporting should state that decoding alone is not proof of vulnerability.

How a recruiter or reviewer should read this evidence

The value of this evidence is not only the screenshot itself. The value is that it demonstrates a structured API security testing process: controlled lab setup, targeted testing, safe interpretation of results, and clear separation between confirmed vulnerabilities and controls that rejected attack attempts.

Finding interpretation

Where the evidence shows vulnerable behaviour, the report should explain the security impact, the affected object or workflow, and the remediation without publishing sensitive raw values.

4. Risk, Impact, and Remediation Guidance

Risk perspective

If JWTs contain excessive sensitive data, lack expiry, use weak signing, or are accepted after tampering, attackers may abuse them to impersonate users or escalate privileges.

Recommended remediation / hardening actions

• Validate JWT signatures server-side.

• Reject alg:none and unexpected algorithms.

• Use short token expiry.

• Avoid storing secrets or sensitive PII in JWT payloads.

• Validate issuer, audience, expiry, and not-before claims.

Validation guidance after remediation

• Repeat the same test workflow after remediation and compare the response behaviour.

• Confirm that sensitive data is no longer exposed in raw API responses.

• Confirm that unauthorized requests return secure error responses such as 401 or 403 where appropriate.

• Confirm that security controls are enforced server-side, not only in the frontend or client collection.

5. GitHub Publication and Redaction Guidance

Sensitive values to redact before public upload

• Full JWT string.

• User IDs and email addresses.

• Roles if tied to real users.

• Issuer/audience values if sensitive.

• Any token signature or secret.

General API evidence redaction checklist

• Blur JWT access tokens, bearer tokens, OAuth codes, cookies, and authorization headers.

• Blur email addresses, phone numbers, user IDs, vehicle IDs, order IDs, and private object identifiers.

• Do not upload Postman environment files containing live variables or secrets.

• Do not upload raw response exports containing personal data or secret-bearing values.

• Use placeholders such as , , and .

GitHub link format for 02-api-security/README.md

```
[JWT decoded in jwt.io](./reports/evidence/03-jwtio-decoded-token-evidence-report.pdf)
```

Final classification

This PDF is suitable for a public GitHub portfolio only after the embedded screenshot has been reviewed and sensitive values are confirmed blurred or removed.